KØBENHAVNS UNIVERSITET

Introduktion til diskret matematik og algoritmer: Exam April 8, 2026 Problems and Solutions

Please note that the main purpose of this document is to explain what the correct solutions are and how to arrive at them. Thus, while the text below is certainly intended to provide good examples of how to solve problems and reason about solutions, these examples do not necessarily specify exactly how the handed-in exams were expected to look like. This is especially so since for many problems there are more than one correct way of solving them. As communicated during the course, the course notes published on Absalon, which contain many problems that we worked out in class, are probably the best indicator of what level of detail is expected from the exam solutions.

1 (60 p) Solve the following problems using techniques that we have learned during the course. Please make sure to provide not just intuition but also formal proofs.

1a (30 p) Prove that if $x_1, x_2, \ldots, x_n$ are non-negative real numbers, then it always holds that

$$(1 + x_1)(1 + x_2) \cdots (1 + x_n) \geq 1 + x_1 + x_2 + \cdots + x_n \,.$$

Solution: We prove this inequality by induction over n.

Base case *($n = 1$):* In this case we have that equality $1 + x_1 = 1 + x_1$ holds between the left-hand side and the right-hand side.

Induction step: Assume as our induction hypothesis that

$$(1 + x_1)(1 + x_2) \cdots (1 + x_n) \geq 1 + x_1 + x_2 + \cdots + x_n \tag{1}$$

holds. By applying (1) to the product $(1 + x_1)(1 + x_2) \cdots (1 + x_n)(1 + x_{n+1})$, for the induction step we get

$$\begin{aligned}
&(1 + x_1)(1 + x_2) \cdots (1 + x_n)(1 + x_{n+1}) && \text{(2a)}\\
&\geq (1 + x_1 + x_2 + \cdots + x_n)(1 + x_{n+1}) && \text{(2b)}\\
&= (1 + x_1 + x_2 + \cdots + x_n) + (x_{n+1} + x_1 x_{n+1} + \cdots + x_n x_{n+1}) && \text{(2c)}\\
&\geq 1 + x_1 + x_2 + \cdots + x_n + x_{n+1} \,, && \text{(2d)}
\end{aligned}$$

where the last inequality holds since all the terms $x_i x_{n+1}$ are non-negative by assumption.

The claim follows by the induction principle.

1b (30 p) Prove that every integer $n \geq 12$ can be written as $4a + 5b$ for a and b non-negative integers.

Solution: We prove this by induction over n.

Base cases: Here we need to consider several base cases:

- For $n = 12$ we have $12 = 4 \cdot 3 + 5 \cdot 0$.
- For $n = 13$ we have $13 = 4 \cdot 2 + 5 \cdot 1$.
- For $n = 14$ we have $14 = 4 \cdot 1 + 5 \cdot 2$.
- Finally, for $n = 15$ we have $15 = 4 \cdot 0 + 5 \cdot 3$.

Induction step: Suppose that $n \geq 16$. Our induction hypothesis is that $n - 4 \geq 12$ can be written as $n - 4 = 4a + 5b$ for $a, b \geq 0$. Then clearly it holds that

$$n = (n - 4) + 4 = 4a + 5b + 4 = 4(a + 1) + 5\,, \tag{3}$$

and so n can also be written on this form.

The claim follows by the induction principle.

2 (60 p) As Jakob was preparing this year's instalment of the IDMA course in early 2026, he realized that he had been teaching discrete mathematics and algorithms in Copenhagen for 6 years. For some reason that is not so easy to explain, this made Jakob wonder what the largest positive integer n is such that $26!/6^n$ is also an integer. Could you help Jakob figure this out? Jakob does not really trust calculators, so he would very much like to see a clean and simple solution by hand that just uses what we learned during the IDMA course.

Solution: The number $26!/6^n$ is an integer precisely when 6^n divides $26! = 26 \cdot 25 \cdot 24 \cdots 3 \cdot 2 \cdot 1$. Since 2 and 3 are prime numbers, for any number M it holds that $6^n | M$ if and only if $2^n | M$ and $3^n | M$. That is, we just need to figure out how many factors of 2 and 3 there are in 26!.

There are $\lfloor 26/3 \rfloor = 8$ positive numbers less than or equal to 26 that are divisible by 3 (namely $3, 6, 9, 12, 15, 18, 21, 24$). Out of these, there are $\lfloor 26/3^2 \rfloor = 2$ numbers that are divisible by 9, and so contribute an extra factor 3 (namely 9 and 18). Since $3^3 = 27 > 26$, no number can contribute more that two factors 3. This means that the largest n such that 3^n divides 26! is $n = 8 + 2 = 10$.

It is easy to check that 2^{10} divides 26!, since enough factors of 2 are contributed already by $2, 4, 6, 8, 10$, and 12.

We can conclude that the largest n such that $26!/6^n$ is an integer is $n = 10$.

3 (80 p) In this problem we consider linked lists. The records in our list datatype have four attributes:

- `data`: the data stored in the record (which we will not care about, as usual);
- `key`: used for sorting the records in the list;
- `prev`: link to the previous element in the list;
- `next`: link to the next element in the list.

The lists are sorted in increasing order based on the key values in `key`. We assume that distinct records in our list have distinct key values, so that every value identifies a unique record in a list. To make the coding easier, every list is initialized with two dummy records `head` and `tail` that do not contain any data but are such that

- `head.key` = `-INFINITY`, `head.prev` = `NULL`, and `head.next` = `tail`;
- `tail.key` = `+INFINITY`, `tail.prev` = `head`, and `tail.next` = `NULL`.

This ensures that `head` will always be the first element in the list since `-INFINITY`$= -\infty$ is smaller than any key value and that `tail` will always be last since `+INFINITY`$= +\infty$ is larger than any key value.

Below you find four pseudocode methods for operating on the list. Please identify which method:

(a) **Inserts** an element identified by a key value in the list.

(b) **Removes** an element identified by a key value from the list.

(c) **Looks up and returns** an element identified by a key value from the list.

Note that there is one spoiler piece of code that does none of the above. All four pieces of code start with exactly the same while loop, and all of them have a return statement regardless of whether this is needed or not (since it would be too easy to distinguish the methods otherwise).

You receive 10 points for each correct match and 10 more points for a brief but convincing explanation why the piece of code does what you claim it does (or, for the spoiler piece of code, why it does something else and why this does not make any sense).

```
METHOD1 (record)
    current := head
    while record.key > current.key {
        current := current.next
    }
    if record.key == current.key {
        previous      := current.prev
        current       := current.next
        previous.next := current
        current.prev  := previous
    }
    return record

METHOD2 (record)
    current := head
    while record.key > current.key {
        current := current.next
    }
    if record.key != current.key {
        current := NULL
    }
    return current

METHOD3 (record)
    current := head
    while record.key > current.key {
        current := current.next
    }
```

```
    if record.key >= current.key {
        previous      := current.prev
        current.prev  := previous.next
        previous      := previous.next
        current.next  := previous
    }
    return current

METHOD4 (record)
    current := head
    while record.key > current.key {
        current := current.next
    }
    if record.key < current.key {
        previous      := current.prev
        previous.next := record
        record.prev   := previous
        record.next   := current
        current.prev  := record
    }
    return record
```

Solution: The four pseudocode methods do the following:

1. `METHOD1` removes an item from the list.
2. `METHOD2` looks up an element in the list.
3. `METHOD3` is a spoiler piece of code that does nothing sensible.
4. `METHOD4` inserts an element in the list.

Here comes a more detailed explanation. As pointed out in the problem statement, all methods have the same initial while loop. This loop finds the first element in the list that has a key that is larger than or equal to the key in the record passed as an argument to the method. In other words, at the end of the while loop, it holds that `current` is the first list element such that `current.key` $\geq$ `record.key`. What this means is the following:

- If there is an element in the list with key value `record.key`, then this element is `current`.
- Otherwise, if `record` is to be inserted in the list, then it should be inserted right before `current`.

Once we have made these observations, it is not too complicated to figure out what the different methods do.

`METHOD1` does not do anything unless `record.key == current.key`, i.e., unless the key value is already there in the list. If the key value is there, the `next` pointer of the element before `current` is changed to point to the element after `current`, and the element after current has its `prev` pointer changed to refer to the element before `current`. In other words, the element `current`, is **removed** from the list.

For `METHOD2`, if `record.key == current.key`, which is the case precisely when an element with key value `record.key` is in the list, then the method returns the record `current`. Otherwise

it returns `NULL`. That is to say that the method locates the list element with key value `record.key` or returns `NULL` if there is no such element, which is a **look-up**.

`METHOD3` also locates the list element with key value `record.key` if such an element exists. But then the method overwrites the `prev` and `next` pointers of this element to point to the element itself, which destroys the list.

`METHOD4`, finally, **inserts** `record` in the correct place right before `current` (unless it holds that `record.key == current.key`, in which case nothing is done since the key is already there in the list). This is achieved by making `previous == current.prev` point to `record` as its next element (with the `prev` pointer of `record` pointing back) and then making `current` point to `record` as its previous element (with the `next` pointer of `record` pointing back).

4 (60 p) For each of the propositional logic formulas below, determine whether it is a tautology or not. If the formula is not a tautology, show how to add a single connective $\wedge$, $\vee$, or $\neg$ to make it into a tautology. Please make sure to justify your answers (e.g., by presenting truth tables, or by using rules for rewriting logic formulas that we have learned in class).

4a (30 p) $\big((p \to q) \to r\big) \vee \big((\neg p \wedge \neg r) \vee (q \wedge \neg r)\big)$

Solution: This formula is a tautology, i.e., it is always true.

To see this, look first at the left-hand side $(p \to q) \to r$ of the disjunction. We know that $A \to B$ is false precisely when A is true and B is false. This means that $(p \to q) \to r$ is false when r is false and $p \to q$ is true. We have that $p \to q$ is true, in turn, when either p is false or q is true. From this we can conclude that whenever the left-hand side $(p \to q) \to r$ is false, it must hold that $(\neg p \vee q) \wedge \neg r$ is true, and using distributivity we can rewrite this to $(\neg p \wedge \neg r) \vee (q \wedge \neg r)$, which is the right-hand side.

We conclude that either the left-hand side or the right-hand side of the disjunction is always true, so the formula is a tautology as claimed.

4b (30 p) $\big(p \to (q \vee r)\big) \leftrightarrow \big((p \vee q) \vee (p \wedge r)\big)$

Solution: This is not a tautology. If we set p and q to false and r to true, we get that $p \to (q \vee r)$ evaluates to true but that $(p \vee q) \vee (p \wedge r)$ evaluates to false, and so the whole formula evaluates to false.

If we change the first p on the right-hand side to $\neg p$, we obtain the formula

$$\big(p \to (q \vee r)\big) \leftrightarrow \big((\neg p \vee q) \vee (p \wedge r)\big) \ , \tag{4}$$

which is a tautology. To see why this is so, recall that we have the equivalence

$$A \to B \ \equiv \ \neg A \vee B \tag{5}$$

since the only way the implication $A \to B$ can be false is that A is true and B is false, as we have learned in the course. We can now rewrite the left-hand side of (4) as

$$\begin{aligned} p \to (q \vee r) \ &\equiv \ \neg p \vee (q \vee r) && \text{(6a)} \\ &\equiv \ (\neg p \vee q) \vee r && \text{(6b)} \\ &\equiv \ (\neg p \vee q) \vee (p \wedge r) && \text{(6c)} \end{aligned}$$

by using the property (5) of implication, associativity of disjunction, and the (slightly stupid) fact that if $\neg p \vee q$ is not already true, then this means that p is guaranteed to be true, so that

the subformulas r and $p \wedge r$ evaluate to the same truth value. (If there is any uncertainty about an argument like this, it is always safe to fall back to truth tables and check that what is claimed is in fact true.)

This line of reasoning show that the left-hand and right-hand sides of of the formula (4) are equivalent, and so the formula will always evaluate to true.

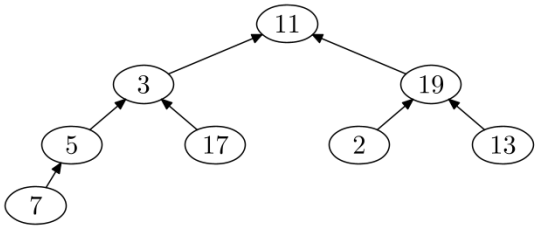
5 (70 p) In this problem we wish to study heaps and how they can be used for sorting.

5a (50 p) Run heapsort by hand to sort the array $A = [11, 3, 19, 5, 17, 2, 13, 7]$ in increasing order and give a detailed explanation of the first few steps of the algorithm. Specifically, show how the heap is built and how the first two numbers are sorted into their correct final positions. Make sure to explain what calls to the heap are made from the sorting algorithm and what comparisons are made, and illustrate how the heap and array have changed at the end of every "heapify" or "bubble" call (after all recursive subcalls have been taken care of).

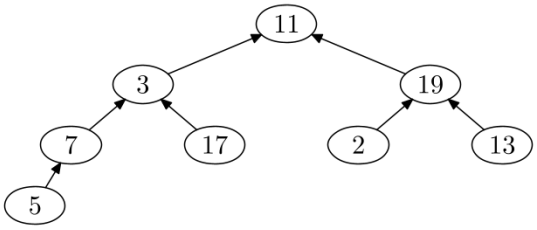
Remark: Please note that there is no need to present the entire execution of the sorting algorithm. Just showing the initial steps as described above is fully sufficient.

Solution: We illustrate in Figure 1 how the heap used for sorting the array is built and then modified as the first two numbers are extracted.

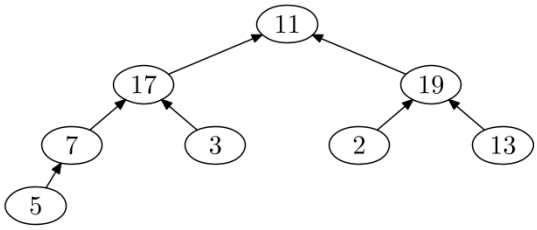
1. The input array is shown in a heap structure in Figure 1a.
2. We need to run BUILD-MAX-HEAP, which builds max-heaps bottom-up using the MAX-HEAPIFY method. The 5th through 8th nodes vacuously constitute max-heaps of size 1, so the first time MAX-HEAPIFY is run is on the 4th node containing 5. Since the child 7 is larger, the elements 5 and 7 swap places as shown in Figure 1b. When MAX-HEAPIFY is called for the subheap rooted at the 8th node, it returns immediately, since that subheap has size 1 and so is a correct max-heap vacuously.
3. MAX-HEAPIFY is then run on the 3rd node, but since 19 is larger than both of its children 2 and 13 nothing changes, and we still have the heap in Figure 1b.
4. For the 2nd node, the number 3 is smaller than the largest child 17, so the two numbers are swapped and the recursive MAX-HEAPIFY call returns immediately, resulting in the heap in Figure 1c.
5. In the final MAX-HEAPIFY call of BUILD-MAX-HEAP, the number 11 at the root is compared to both children of the root. The largest child 19 is moved to the root and 11 is shifted down to the 3rd node. In the first recursive MAX-HEAPIFY call, 11 is compared to the children 2 and 13, and the largest child 13 is moved up and 11 is shifted down to the 7th node. The final recursive call for the 7th node terminates since the subheap has size 1. The max-heap resulting from the BUILD-MAX-HEAP call is shown in Figure 1d.
6. The heapsort algorithm now repeatedly removes the root, places the last element in the heap in the root position, and runs MAX-HEAPIFY. Right after 19 has been removed and 5 has been put in the root position, we have the heap structure in Figure 1e. A sequence of recursive MAX-HEAPIFY calls compare 5 first to the largest of the two children 17 and 13 of the root, moving up 17 and moving down 5 to the 2nd node, then to the largest of the two children 7 and 3, moving up 7 and moving 5 further down to the 4th node. This yields the max-heap on seven elements in Figure 1f.



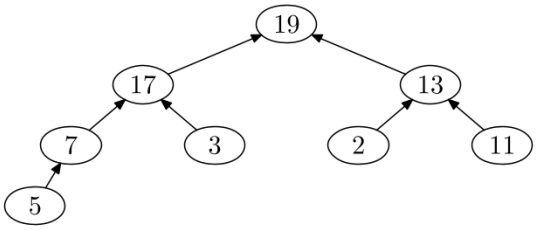
(a) Input array.



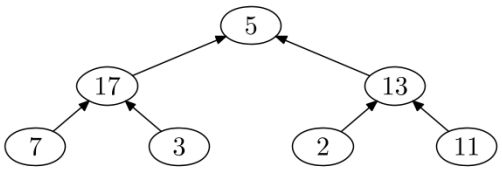
(b) After max-heapify on 4th node.



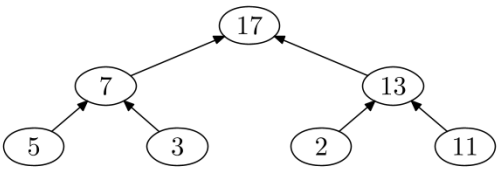
(c) After max-heapify on 2nd node.



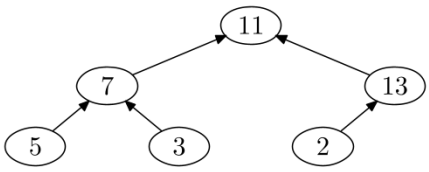
(d) After max-heapify on root node.



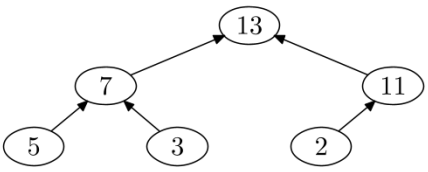
(e) Right after removing largest element 19.



(f) After max-heapify on root node.



(g) Right after removing largest element 17.



(h) After max-heapify on root again.

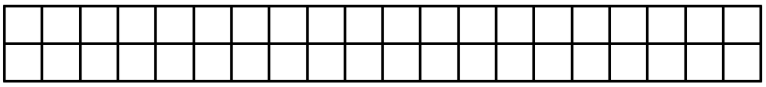
Figure 1: Heap configurations for heap sort in Problem 5a.

7. Right after 17 has been removed and 11 has been put in the root position, we have the heap structure in Figure 1g. The recursive MAX-HEAPIFY calls compare 11 to the largest of the two children 7 and 13 of the root, changing places of 11 and 13, but then we terminate since 11 is larger than 2. The resulting max-heap on six elements is depicted in Figure 1h.

This completes the description of how the first two numbers 19 and 17 are extracted from the heap during heapsort, which is what we were asked to explain and illustrate.

5b (20 p) Jakob thinks he has figured out a nifty way of finding the *minimal* element in a max-heap by calling the following recursive algorithm with index $i = 1$:

```
MAX-HEAP-FIND-MIN (A, i)
    if (2 * i > A.heapsize) {
        smallest := i
    }
```



(a) Example 2×20 grid.

(b) Only tiling of 2×1 grid.

(c) First tiling of 2×2 grid.

(d) Second tiling of 2×2 grid.

(e) First tiling of 2×3 grid.

(f) Second tiling of 2×3 grid.

(g) Third tiling of 2×3 grid.

Figure 2: Tilings of $2 \times n$ grids in Problem 6.

```
else {
    smallest := 2 * i
}
if (2 * i + 1 <= A.heapsize AND A[2 * i + 1] < A[smallest]) {
    smallest := 2 * i + 1
}
if (smallest == i) {
    return (A[i])
}
else {
    return (MAX-HEAP-FIND-MIN (A, smallest))
}
```

Is Jakob right that this will always return the smallest element (assuming that A is a correct max-heap)? Please provide either a proof of correctness or a counterexample where the input is a max-heap but `MAX-HEAP-FIND-MIN` fails to find the minimal element.

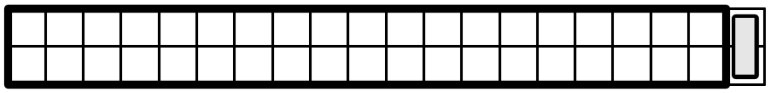
Solution: No, this is not true. The algorithm recursively picks the smallest child and descends into that subtree, but there is no reason why the globally smallest element should reside in this subtree. For counter-examples, see, e.g., the heaps in Figures 1f and 1h.

6 (80 p) In this problem we wish to count the number of ways of tiling a $2 \times n$ grid (as in Figure 2a) with 2×1 tiles so that every cell in the grid is covered by exactly one tile. There is 1 such tiling for a 2×1 grid (Figure 2b); exactly 2 different tilings for a 2×2 grid (Figures 2c and 2d); and exactly 3 different tilings for a 2×3 grid (Figures 2e, 2f, and 2g).

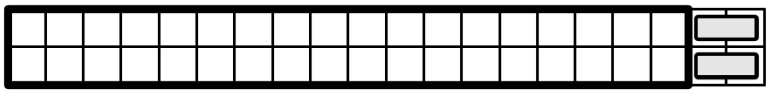
6a (20 p) How many different tilings are there for a 2×4 grid? Make sure to explain how you compute this number so that we can follow your reasoning.

Solution: Do a case analysis for the right-most tile(s).

If the right-most tile is a vertical 2×1 tile, then this tile can be combined with the tilings in Figures 2e, 2f, and 2g of the first 3 columns to yield 3 solutions.



(a) Case 1 in case analysis with vertical tile plus tilings of $2 \times (n-1)$ grid.



(b) Case 2 in case analysis with horizontal tiles plus tilings of $2 \times (n-2)$ grid.

Figure 3: Case analysis for tilings of $2 \times n$ grids in solution of Problem 6.

If the right-most tiles are two horizontal 1×2 tiles on top of one another, then these two tiles can be combined with the tilings Figures 2c and 2d of the first 2 columns to yield 2 more solutions.

We can never get the same solution from the two difference cases, since the right-most end of the tiling is different in the two cases. All in all, we therefore get a total of $2+3=5$ different tilings for the 2×4 grid.

6b (60 p) How many different tilings are there for a $2 \times n$ grid for a positive integer n?

Hint: Find a nice formula for this number in terms of concepts we learned in class.

Solution: Looking at the case analysis we did for Problem 6a, we see that the number of tilings seem to behave like the Fibonacci numbers. In one sentence, all we need to do to prove this is to turn the solution of Problem 6a into the inductive step in an induction proof.

In a bit more detail, let us write write T_n to denote the number of different tilings of a $2 \times n$ grid. We claim that $T_n = F_{n+1}$, where F_n denotes the Fibonacci numbers defined by $F_1 = F_2 = 1$ and $F_n = F_{n-1} + F_{n-2}$ for $n \geq 3$. Let us prove this by induction over n.

Base cases: We have seen from the case analysis in the problem statement that we have

$$T_1 = 1 \tag{7a}$$

$$T_2 = 2 \tag{7b}$$

and

$$T_3 = 3\ , \tag{7c}$$

which shows that $T_n = F_{n+1}$ for $n = 1, 2, 3$.

Induction step: Suppose as our induction hypothesis that $T_{n-1} = F_n$ and $T_{n-2} = F_{n-1}$, and consider T_n. We do a case analysis for the right-most tile(s) in the grid.

If the right-most tile is a vertical 2×1 tile (as in Figure 3a), then the rest of the grid can be tiled in T_{n-1} ways. If the right-most tiles are two horizontal 1×2 tiles on top of one another (as in Figure 3b), then the rest of the grid can be tiled in T_{n-2} ways. There are no overlaps between these two different types of tilings, since the right-most column is covered by a vertical tile in all tilings in the first case in Figure 3a and by two horizontal tiles in the second case in Figure 3b. Therefore, the total number of tilings is $T_n = T_{n-1} + T_{n-2} = F_n + F_{n-1} = F_{n+1}$ (using the induction hypothesis for T_{n-1} and T_{n-2}).

It now follows by the principle of mathematical induction that the number of different tilings T_n of a $2 \times n$ grid is the $(n+1)$st Fibonacci number F_{n+1}.

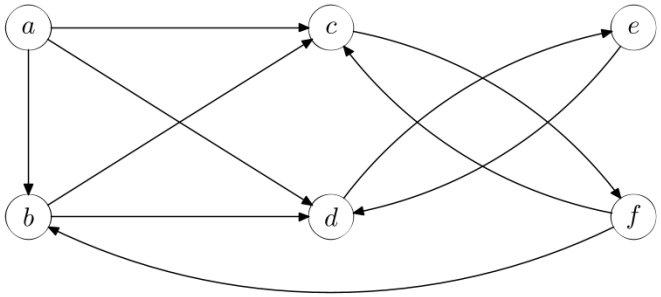


Figure 4: Graph for Problem 7

7 (80 p) This problems is about relations in general and graphs in particular.

7a (10 p) Say that the relation $E(u, v)$ holds for two vertices u and v in a directed graph if there is an edge from u to v. Present the matrix representation of E for the graph in Figure 4.

Solution: We identify the rows and columns $1, 2, \ldots, 6$ with the vertices $a, b, \ldots, f$ in that order, and then add a 1 in the matrix in every position (i, j) such that there is an edge from i to j in the graph, which yields the matrix

$$M_E = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \end{pmatrix} \tag{8}$$

for the graph in Figure 4.

7b (10 p) Take the reflexive closure R of the relation E in Problem 7a and present the matrix representation of this relation.

Solution: The reflexive closure R of the relation E is the smallest relation $R \supseteq E$ such that $(i, i) \in R$ holds for all i. We obtain the reflexive closure of E by adding all such pairs, i.e., by setting $R = E \cup \{(i, i) \mid i \in \{a, b, c, d, e, f\}\}$. In terms of matrix representations, this corresponds to adding 1s along the diagonal, yielding the matrix

$$M_R = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{pmatrix} \tag{9}$$

(which, in terms of graphs, corresponds to adding self-loop edges to all vertices).

7c (20 p) Take the transitive closure T of the relation R in Problem 7b and present the matrix representation of this relation.

Solution: The transitive closure T of R is the smallest relation $T \supseteq R$ such that if $(i,j) \in T$ and $(j,k) \in T$ both hold, then $(i,k) \in T$ also holds.

If we use that the relation R is a directed graph (with self-loops), then we see that the transitive closure is just the relation T such that $(i,j) \in T$ if there is a path from i to j in the graph (or if $i = j$ is one and the same vertex). If we go over all ordered pairs of vertices (i,j) and add a 1 in position (i,j) in the matrix precisely when there is a path from i to j, it can be verified that this yields the matrix representation

$$M_T = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \tag{10}$$

for the transitive closure T of R.

7d (40 p) Now consider a new relation $S(u,v)$ where we define $S(u,v)$ to hold for two vertices u and v precisely when $T(u,v)$ and $T(v,u)$ both hold for the relation T defined as in Problem 7c.

This gives a particular type of relation that we discussed in class. What is this type of relation called? Explain, briefly but clearly, why we are guaranteed to always get this type of relation when performing the steps described in Problems 7b, 7c, and 7d.

Also, this relation S groups the vertices of the graph in a particular way. What is the technical term that we use for such groups of vertices in a graph?

Remark: Note that you do *not* have to present any matrix representation for this relation S.

Solution: Although this is not necessary (as explained in the problem statement), let us start by giving the matrix representation

$$M_S = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{pmatrix} \tag{11}$$

just to be helpful. We see that this partitions the vertex set into the subsets $\{a\}$, $\{b,c,f\}$, and $\{d,e\}$, where in each subset all pairs of vertices are related. In other words, S is an equivalence relation, i.e., a relation that is reflexive, transitive, and symmetric.

To see why this is guaranteed to be the case, note that T is the reflexive and transitive closure of E, so T is definitely reflexive and transitive by construction. This relation is not symmetric, however. But S is defined as $S = T \cap T^{-1}$, which means that S is a symmetric relation by construction. (If $(i,j) \in S$, then by definition both $(i,j) \in T$ and $(i,j) \in T^{-1}$ hold, and the second property is just the same thing as saying $(j,i) \in T$, from which it again follows by the definition of S that $(j,i) \in S$.) Also, it is straightforward to verify directly from the definitions that if T is a transitive relation, then so is $T \cap T^{-1}$, and the same obviously holds for reflexivity. This shows that S is guaranteed to be an equivalence relation.

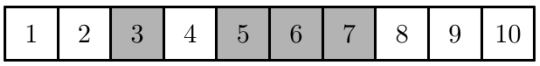


Figure 5: Illustration of computed set S in Problem 8a.

Recall that we observed in our solution to Problem 7c that $(i, j) \in T$ if and only if there is a path from i to j in the graph. Since $S = T \cap T^{-1}$, we have that $(i, j) \in S$ holds if and only if there are paths in both directions between i and j in the graph, i.e., if the two vertices are in the same *strongly connected component*, which is the technical term we are looking for. We can conclude that the relation S partitions the vertices of the graph into the strongly connected components $\{a\}$, $\{b, c, f\}$, and $\{d, e\}$.

8 (100 p) Suppose that we have a universe of n elements and are interested in counting the number of multisets of size r. Jakob claims to have figured out that the number of such multisets is $\binom{n+r-1}{n-1}$. His idea is to prove this by using the following algorithm to translate any given size-r multiset M of $[n]$ to a standard subset S of $[n + r - 1]$ of size $n - 1$:

```
MULTISET2SET (M)
    elem := 1
    S    := empty-set
    for i := 1 upto n - 1 {
        c := number of copies of element i in M
        for j := 1 upto c {
            elem := elem + 1
        }
        S := S union {elem}
        elem := elem + 1
    }
    return S
```

8a (20 p) Suppose that we have $n = 5$ elements in the universe and are interested in multisets of size $r = 6$. Describe what subset S is returned when the algorithm `MULTISET2SET` above is run with the multiset $M = [1, 1, 2, 5, 5, 5]$ as input. Make sure to explain, briefly but clearly, how this set is computed.

Solution: `MULTISET2SET` starts by setting `elem` $= 1$ and $S = \emptyset$.

Since there are two copies of element 1 in M, in the first iteration of the for loop `elem` is incremented to 3, after which 3 is added to S. Finally `elem` is incremented again to 4.

In the second iteration of the for loop, `elem` is incremented to 5 since there is one copy of element 2 in M. After this, 5 is added to S and `elem` is incremented again to 6.

Since there are no copies of the elements 3 or 4 in M, the next two iterations add 6 and 7 to S, and then the for loop ends (since it only runs to $n - 1 = 4$). Hence, the set that is returned for input $M = [1, 1, 2, 5, 5, 5]$ is $S = \{3, 5, 6, 7\}$, as illustrated in Figure 5.

8b (40 p) Is Jakob's new algorithm `MULTISET2SET` correct? Can you prove that the algorithm provides a translation that shows that the number of multisets of size r from a universe of size n is exactly $\binom{n+r-1}{n-1}$? Or if there is an error in Jakob's algorithm, then what is it?

Hint: A proof of correctness would need to go through essentially the same kind of steps that we did in class. If Jakob's method is in fact incorrect, then just one small counterexample is sufficient.

Solution: Yes, the algorithm `MULTISET2SET` is correct in that it translates multisets of size r from a universe of size n to standard subsets of size $n-1$ from a universe of size $n+r-1$. (Note that in class we translated to standard subsets of size r instead, but this is no big difference since the equality $\binom{n+r-1}{r} = \binom{n+r-1}{(n+r-1)-r} = \binom{n+r-1}{n-1}$ holds.)

The intuition is that the chosen elements in S tell us when we should skip to the next element in the multiset M. If we look at the set $S = \{3, 5, 6, 7\}$ in Figure 5, then we can reconstruct the multiset $M = [1, 1, 2, 5, 5, 5]$ as follows:

- Include two copies of 1 and then shift to 2 because of the chosen element/box 3.
- Include one copy of 2 and then shift to 3 because of the chosen element/box 5.
- Skip 3 and shift to 4 because the next box 6 is chosen.
- Skip 4 and shift to 5 because the next box 7 is also chosen.
- Finally, include three copies of 5 corresponding to the last three boxes.

But we need to argue this a little bit more formally.

First, we claim that two different multisets M and M' must be translated to different subsets S and S'. To see this, look at the smallest element i such that the number of copies of i in M and M' are different. At that point, the algorithm `MULTISET2SET` will select different elements for S and S', and since `elem` is always increased this means that S and S' will stay different after this point. This shows that the translation function computed by `MULTISET2SET` is injective.

To argue that the translation function is surjective, we describe how we can invert the function to translate any subset S back to a multiset M. Briefly, the inverse translation is as follows (where it might be helpful to refer to the illustration in Figure 5 again):

- The leftmost number of unchosen elements/boxes tells us how many elements 1 we have in the multiset. If box number 1 is chosen, i.e., if $1 \in S$, then there are no copies of the element 1 in the multiset M that we construct.
- We then look at the number of unchosen elements/boxes between each consecutive pair of chosen boxes, and this is the number of copies of the next element i for $i = 2, 3, \ldots, n-1$. If there are two chosen boxes next to each other, then this means there is no copy of the corresponding element in M.
- Finally, the rightmost number of unchosen elements/boxes shows the number of copies of the element n in M.

Since there are $n-1$ chosen elements/boxes in S, all elements added to M in this way are between 1 and n. Since the total number of unchosen boxes is $n + r - 1 - (n-1) = r$, the constructed multiset M will contain r elements. It is straightforward to verify that if we run the translation function `MULTISET2SET` on this multiset M, then the result is exactly the set S that we started with. This shows that the translation function is surjective.

Since the translation function `MULTISET2SET` is both injective and surjective, which is to say it is bijective, this shows that the number of multisets of size r from a universe of size n is exactly $\binom{n+r-1}{n-1}$.

8c (40 p) Consider the original translation from multisets of size r from a universe of size n to subsets of size r from a universe of size $n+r-1$ that we used in class, and that we proved to be correct. For fixed positive integers n and r, let us say that the Boolean predicate

$T_{n,r}(M, S)$ is true if the multiset M can be translated to the standard subset S by the method we studied in class, and that $T_{n,r}(M, S)$ is false otherwise.

Please indicate for all alternatives below whether $T_{n,r}$ satisfies this property or not. For each property, you get points for the correct answer, a deduction for a wrong answer, and no change in your score if you do not indicate an answer for a specific property.

1. $T_{n,r}$ describes a relation.
2. $T_{n,r}$ describes a function.
3. $T_{n,r}$ is surjective.
4. $T_{n,r}$ is reflexive.
5. $T_{n,r}$ is a partial order.
6. $T_{n,r}$ is injective.
7. $T_{n,r}$ is symmetric.
8. $T_{n,r}$ is an equivalence relation.

Solution: The translation will convert any multiset into exactly one set, so $T_{n,r}$ is a function (and therefore also a relation). We proved in class that this function is injective and surjective.

Asking about partial orders and equivalence relations only makes sense for relations $R \subseteq A \times A$ on a single set A and not for relations $R \subseteq A \times B$ from a set A to another set B. Therefore, $T_{n,r}$ is *not* a partial order or equivalence relation, and for the same reason it is not reflexive or symmetric.

Summarizing, we obtain the following answers:

1. $T_{n,r}$ describes a relation. **YES**
2. $T_{n,r}$ describes a function. **YES**
3. $T_{n,r}$ is surjective. **YES**
4. $T_{n,r}$ is reflexive. ***NO***
5. $T_{n,r}$ is a partial order. ***NO***
6. $T_{n,r}$ is injective. **YES**
7. $T_{n,r}$ is symmetric. ***NO***
8. $T_{n,r}$ is an equivalence relation. ***NO***

9 (120 p) Consider the graph in Figure 6 and the following ordered sequences listing the vertices in this graph:

1. a, d, b, c, e, f, g, h.
2. a, d, c, f, b, e, g, h.
3. a, d, c, b, e, g, h, f.
4. a, d, b, c, e, g, f, h.

For each of the sequences above, determine whether it can be the result of:

(a) a breadth-first search with vertices listed in order of visits;
(b) a depth-first search with vertices listed in order of discovery;
(c) a shortest-path computation with vertices listed in the order they are removed from the priority queue.

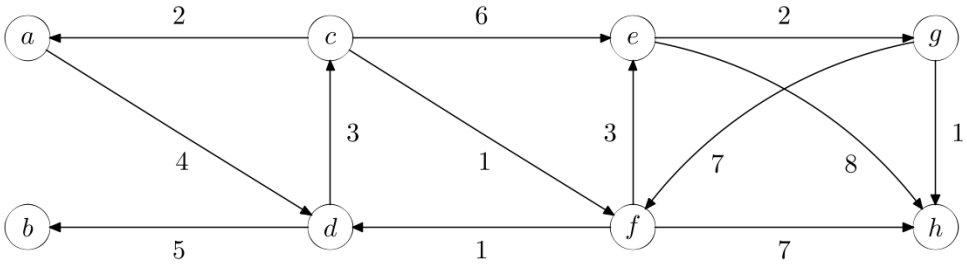


Figure 6: Graph for Problem 9

Each sequence above is the output of at most one of the algorithms, but since there are four sequences there could be a sequence that cannot be produced by any of the algorithms.

Partial credit is given for matching algorithms and vertex sequences correctly. For full credit, you need to also give a brief overview of how and why the algorithms process the vertices in the given order (such as explaining the order of recursive calls, edges processed, or similar), or why none of the proposed algorithms could yield the sequence in question, but you do not have to provide detailed information about any auxiliary data structures used in the algorithms.

Solution: Let us consider the different algorithms in the order listed and try to find sequences that would match their outputs. Since we are promised that each sequence is the output of at most one algorithm, we are done with each sequence as soon as we find a match.

Sequence 1 is the result of breadth-first search starting with vertex a. To see this, consider how the different vertices are enqueued in and dequeued from the queue if we start the breadth-first search in vertex a:

Dequeued vertex	**Enqueued vertices**	**Queue**
—	—	(a)
a	$\{d\}$	(d)
d	$\{b, c\}$	(b, c)
b	—	(c)
c	$\{e, f\}$	(e, f)
e	$\{g, h\}$	(f, g, h)
f	—	(g, h)
g	—	(h)
h	—	$()$

Sequence 2 is the result of a call to Dijkstra's algorithm computing shortest paths from vertex a. To see this, consider in which order the vertices are dequeued from the priority queue and how vertex key values are updated as edges are relaxed:

Dequeued	**Updates**	**Priority queue (after relaxations)**
—	—	$\{(a:0), (b:\infty), (c:\infty), (d:\infty), (e:\infty), (f:\infty), (g:\infty), (h:\infty)\}$
a	$\{d\}$	$\{(d:4), (b:\infty), (c:\infty), (e:\infty), (f:\infty), (g:\infty), (h:\infty)\}$
d	$\{b, c\}$	$\{(c:7), (b:9), (e:\infty), (f:\infty), (g:\infty), (h:\infty)\}$
c	$\{e, f\}$	$\{(f:8), (b:9), (e:13), (g:\infty), (h:\infty)\}$
f	$\{e, h\}$	$\{(b:9), (e:11), (h:15), (g:\infty)\}$
b	—	$\{(e:11), (h:15), (g:\infty)\}$
e	$\{g\}$	$\{(g:13), (h:15)\}$
g	$\{h\}$	$\{(h:14)\}$
h	—	$\{\}$

Sequence 3 is a spoiler sequence that cannot be the output of any of the algorithms listed. To see this, consider the following case analysis:

- The sequence cannot be the output of Dijkstra's algorithm, since the vertices are not listed in increasing order of distance. Starting with a, d, c is in order, but b should not be listed before f since vertex b is further away from a than vertex f is.
- A depth-first search starting with a, d, c would next visit an undiscovered neighbour of c and not b.
- In a breadth-first search starting with a, d, c, b, e, the next element would then be the other neighbour f of c and not g.

Sequence 4, finally, is the result of depth-first search starting with vertex a. Below follows an illustration of in which orders vertices are discovered and finished:

Discover a — undiscovered neighbour d
Discover d — undiscovered neighbours b, c
Discover b — no undiscovered neighbours
Finish b, since no undiscovered neighbours
Discover c — undiscovered neighbours e, f
Discover e — undiscovered neighbours g, h
Discover g — undiscovered neighbours f, h
Discover f — undiscovered neighbour h
Discover h — no undiscovered neighbours
Finish h, since no undiscovered neighbours
Finish f, since no undiscovered neighbours left
Finish g, since no undiscovered neighbours left
Finish e, since no undiscovered neighbours left
Finish c, since no undiscovered neighbours left
Finish d, since no undiscovered neighbours left
Finish a, since no undiscovered neighbours left